

Column-Based NoSQL Databases

Sure! Let's dive into the concept of column-based NoSQL databases in simple terms.

Column-based NoSQL databases are like a well-organized library where books (data) are stored in a way that makes it easy to find specific information quickly. Instead of keeping all the information about a book (like title, author, and genre) in one row, these databases store related information in columns. This means that if you want to find out about a specific book, you only need to look at the relevant columns, making the search faster and more efficient.

For example, imagine you have a library where each book has its own shelf (row) but the information about the books is organized by categories (columns) like "Title," "Author," and "Publication Year." If you want to find all the books by a certain author, you can quickly check the "Author" column without having to sift through every single book. This organization is especially useful when dealing with large amounts of data, like tracking user behavior in an online store or analyzing data from IoT devices.

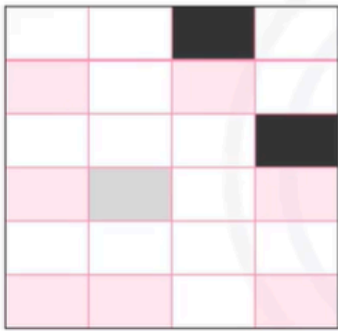
Column based NoSQL database architecture



- Spawned from Google's 'Bigtable'
- Bigtable clones, columnar, or wide-column databases
- Store data in columns or groups of columns

- Column-based database is spawned from an architecture that Google created called Bigtable. These databases are also commonly referred to as Bigtable clones, columnar databases, or wide-column databases. As you can tell from the name, these databases focus on columns and groups of columns when storing and accessing data.

Column based NoSQL database architecture



- Column 'families' are several rows with unique keys belonging to one or more columns
 - Grouped in families as often accessed together
 - Rows in a column family are not required to share the same columns
 - Can share all, a subset, or none
 - Columns can be added to any number of rows or not
-
- Let's learn more about this architecture. The column named families consists of several rows. Each row has a unique key or identifier that belongs to one or more columns. These columns are grouped together in families because they are often accessed together. Rows in a column family are not required to share any of the same columns. Rows can share all columns, a subset of columns, or none of the columns, and columns can be added to some rows and not to others.

Technical advantages: Suitable use cases

- Great for large amounts of sparse data
 - Column databases
 - Handle deployment across clusters of nodes
 - Used for event logging and blogs
 - Counters are a unique use case for column databases
 - Columns can have a time to live (TTL) parameter, making them useful for data with an expiration value
-
- Let's look at some typical use cases for a column-based NoSQL database. Column-based databases are great for when you're dealing with large amounts of sparse data. When compared to row-oriented databases, column-based databases can better compress data and save storage space.
 - In addition, these databases continue the trend of horizontal scalability. As with key value and document databases, column-based databases can handle being deployed across clusters of nodes. Like document databases, a column-based NoSQL database can be used for event logging and blogs, but the data would be stored in a different fashion. For enterprise logging, every application can write to its own set of columns and have each row key formatted in such a way to promote easy lookup based on application and timestamp.
 - Counters are a unique use case for column-based databases. You may come across applications that need an easy way to count or increment as events occur. Some column-based databases, like Cassandra have special column types that allow for simple counters. In addition, columns can have a time-to-live parameter, making them useful for data with an expiration date or time like trial periods or ad timing.

Technical disadvantages: Unsuitable use cases

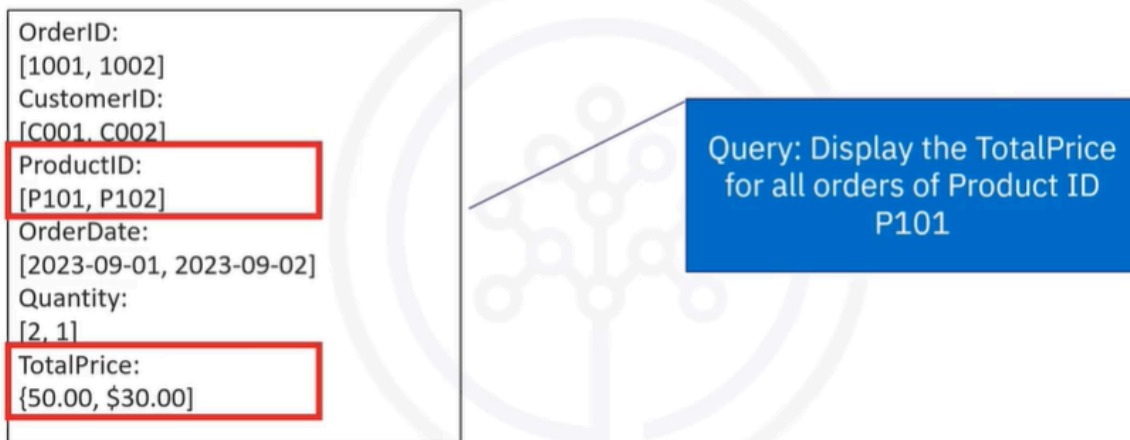
- Traditional ACID transactions
 - Reads and writes are only atomic at the row level
 - In early development, query patterns can change and require numerous changes to column-based databases
 - Can be costly
 - Can slow down the development timeline
- On the other hand, column-based databases should be avoided if you require traditional asset transactions provided by relational databases. Reads and writes are only atomic at the row level. And early into development, query patterns may change and require numerous changes to the column-based designs. This can be costly and slow down the development timeline.

Use cases: Corporate data warehousing



- In a typical data warehousing scenario, you need to collect and store data from different sources, including research such as scientific research, land and business assets, user behavior, and ecommerce data. Row-oriented databases in data warehouses store a record or row data in contiguous blocks, while the column database stores the record using contiguous columns. This data storage method facilitates faster access to the data, enhancing an organization's business intelligence and reporting capabilities.

Use case: E-commerce



- Let's examine how the company would store ecommerce data in a column-oriented database. In this simplified example, you can see each column or key name and its data. And when doing a query such as display the total price for all orders of product ID P101, the query only needs to read the product ID and the total price column data. Next, examine how data analysts can use columnar databases for their work.

Use case: Financial analysis using OLAP



- Financial analysis, online analytical processing, called OLAP, happens when analyzing data that doesn't change often. When performing OLAP, you are working with a large number of records, but only analyzing a subset of the data stored within the columns. For example, analyzing data to build a histogram of insurance premiums paid during the last financial year.

Use case: IoT sensor data



- IoT devices are everywhere, you'll find these devices in cars, trucks, cameras, smartphones, and even light bulbs and luggage, capturing data on demand continuously or periodically. Storing this data in a traditional database requires a lot of data storage space, and data access can suffer latency. Because IoT data is often used for real time and near real-time analysis, including visualizations, data analysts do not want to incur long wait times when querying data. Columnar databases help resolve these problems, helping to provide efficient storage and timely access to IoT data.

Use case: IoT sensor data



Use case: IoT sensor data



- Let's focus on the example of IoT data generated by a commercial truck. These trucks report their speed, geographical location, braking data, automotive health information, images of the driver, dash cam images, and parking camera images back to the company for whom they work, either in almost real time or several times a day. Companies retrieve this data to create custom reports that analyze individual driver performance, extract operation metrics such as fleet need analysis, provide predictive analytics to forecast logistic needs, and perform compliance reporting, including shareholder and tax reporting.

Popular NoSQL database vendors



Recap

In this video, you learned that:

- Column-based databases were spawned from the architecture of Google's Bigtable storage system
- Column-based databases store data in columns or groups of columns
- Column 'families' are several rows, with unique keys, belonging to one or more columns
- Use cases for column-based databases include data warehousing, analytics, business intelligence, and IoT event logging

How can you apply column-based databases in business?

Column-based NoSQL databases can be applied in various business scenarios effectively. Here are two key applications:

1. Data Warehousing and Analytics:

- Businesses often need to analyze large volumes of data from different sources. Column-based databases excel in this area because they store data in a way that allows for faster querying and analysis.
- For example, an e-commerce company can use a column-based database to store sales data, customer information, and product details. When running reports to analyze sales trends, the database can quickly retrieve only the necessary columns, such as "Total Sales" and "Product ID," improving performance and efficiency.

2. IoT Data Management:

- With the rise of IoT devices, businesses collect vast amounts of data from sensors and devices. Column-based databases are well-suited for storing this type of data due to their ability to handle sparse data efficiently.
- For instance, a logistics company can use a column-based database to store real-time data from trucks, such as speed, location, and fuel consumption. This allows the company to analyze driver performance and optimize routes without experiencing delays in data retrieval.

These applications highlight how column-based NoSQL databases can enhance data management and analysis in business settings.

What challenges might arise when using column-based databases in business?

Using column-based databases in business can present several challenges. Here are two key challenges to consider:

1. Limited Transaction Support:

- Column-based databases typically provide atomicity only at the row level, which means that complex transactions involving multiple rows or tables may not be fully supported. This can be a limitation for businesses that

require traditional asset transactions, such as banking or financial applications, where data integrity is crucial.

2. Design Complexity:

- Early in the development process, the design of a column-based database may need to be adjusted as query patterns change. This can lead to increased complexity and costs, as modifying the database structure to accommodate new requirements can be time-consuming and may slow down the development timeline.

These challenges highlight the importance of carefully evaluating the specific needs of a business before implementing a column-based database.

What is the main limitation of transaction support in column-based databases?

The main limitation of transaction support in column-based databases is that they typically provide **atomicity only at the row level**. This means:

- **Limited Scope:** Transactions that involve multiple rows or multiple tables cannot be executed as a single atomic operation. If a transaction fails, it may leave the database in an inconsistent state because not all changes can be rolled back.
- **No Multi-Row Transactions:** Unlike traditional relational databases that support complex transactions across multiple rows and tables, column-based databases may struggle with operations that require consistency across different data points.

This limitation can be problematic for applications that require strict data integrity and consistency, such as financial systems or applications that manage critical business processes.